

How should Metro E-Commerce Logistics audit fulfillment data and establish distributional performance baselines?

Your guide for the lab. Start with the email, then work each card in order. Everything else is optional reference.

Contents

The request

At a glance

Before class

Part A: Can we trust the order file?

Part B: What does a typical order look like?

Part C: Which orders are unusually slow?

Part D: Is Hub C really different?

Take-home: Should we penalize Hub C, and what should the SLA say?

Submit

Checklist

Grading matrix

Memo template

Optional practice

The request

Subject: **Before we penalize Hub C: I need a latency baseline I can defend**



VP of Supply Chain Logistics

To: Lead Operations Analytics Consultant (you)

Reply by Tue, Sep 22 · 11:59 PM ET

Hi Operations Analytics team,

Leadership wants to penalize our regional carrier at Hub C and rewrite our delivery SLAs. Before we do either, I need to trust the order file and know which latency number is fair to put in a contract. Please give me evidence with units and denominators, and tell me plainly what the data can't show.

Specifically, I need answers to 5 questions:

1. Can we trust the order file?

Part A

Our systems write `-1` when age is unknown and `999` when latency is unknown. How many of each are in the 10,000 orders, and which fields can we do arithmetic on? Please don't change or delete any source rows.

2. What does a typical order look like?

Part B

Give me the mean, the median and the spread of valid fulfillment latency, in minutes. If the mean and the median disagree, tell me what that means for a typical customer.

3. Which orders are unusually slow?

Part C

Use a standard rule, not gut feel, to mark the slow tail. How many orders cross the line? Before anyone acts on a flag, I need to know what a flag does and doesn't tell us.

4. Is Hub C really different?

Part D

Compare Hub A, Hub B and Hub C using the same rules: order counts, medians and spread. Make sure the hub counts add up to the total.

5. Should we penalize Hub C, and what should the SLA say?

Take-home

Tell me whether the mean, the median or the 95th percentile should anchor the SLA, and whether the records support a penalty. Tell me what is still uncertain and what evidence we should collect next. Don't claim a hub caused delays from summaries alone.

Please send me:

- your completed workbook, and

- a one-page memo that answers the penalty and SLA questions first, cites every number with its unit (minutes or records) and denominator, and discloses any AI use.

The order extract and the workbook are attached. Thanks. The carrier review depends on this.

Best,

VP of Supply Chain Logistics

Metro E-Commerce Logistics



orders_story.csv



LAB01_Learning_Workbook.xlsx

Your lab at a glance

Before class

setup

untimed

Part A

6 min

Slide 26 → 27

Part B

7 min

Slide 45 → 46

Part C

6 min

Slide 67 → 68

Part D

7 min

Slide 86 → 90

Take-home

60 min

At home

Submit

checklist

Due Tue, Sep
22

Before class: get ready (untimed)

1 Unzip `LAB01_Student_Package.zip` into one folder.

Keep every file together. SPSS looks for the CSV files next to the syntax.

2 Open `LAB01_Learning_Workbook.xlsx` and go to the **Start Here** sheet.

Type your first name, last name and NetID (the part before @nyu.edu). Use **File > Save As** with the workbook name it shows.

3 Open `LAB01_SPSS.sps` in SPSS and fix the first line.

Replace `CHANGE_TO_YOUR_EXTRACTED_LAB_FOLDER` with the path of your unzipped lab folder (the folder, not the `.sps` file; see [Find your folder path](#)). Save As the `.sps` name shown on Start Here.

4 Select the whole **H0** block, then choose **Run > Selection**.

The Output Viewer opens with **H0 Import and Data Screen**.

✅ **Ready when:** the H0 output shows the expected case count and your workbook is saved under your own name.

⚠️ **Stuck?** Don't start Part A on the timer. Ask for help first, or see the SPSS environment guide.

Find your folder path (for the `CD` line)

SPSS needs the path of the **folder** you unzipped, the one that holds `LAB01_SPSS.sps` and the CSV files. It is **not** the path to the `.sps` file. If you open the folder and see the `.sps` file and the CSV files inside, that is the right folder.

Windows

1. Unzip first: right-click `LAB01_Student_Package.zip` > **Extract All** > **Extract**. Do not work inside the zip.
2. Open the extracted `LAB01_Student_Package` folder. You should see `LAB01_SPSS.sps` and the CSV files. If it only holds another folder with the same name, open that one.
3. Click once in the address bar at the top of File Explorer: it turns into the path. Press **Ctrl+C**.
4. Or: go up one level, hold **Shift**, right-click that folder > **Copy as path**, then delete the double quotes it adds.

Looks like

```
C:\Users\you\Downloads\LAB01_Student_Package
```

Mac

1. Unzip first: double-click `LAB01_Student_Package.zip` in Finder.
2. Open the extracted `LAB01_Student_Package` folder. You should see `LAB01_SPSS.sps` and the CSV files. If it only holds another folder with the same name, open that one.
3. Go up one level and click that folder **once** to select it (the folder, not a file inside it). Press **Option+Command+C** (Copy as Pathname), or right-click it, hold **Option** and choose **Copy ... as Pathname**.

Looks like

```
/Users/you/Downloads/LAB01_Student_Package
```

Paste it between the single quotes on the `CD` line:

```
CD 'C:\Users\you\Downloads\LAB01_Student_Package'.
```

* *Windows*

```
CD '/Users/you/Downloads/LAB01_Student_Package'.
```

* *Mac*

⚠ **Common mistakes:** the path ends in `/LAB01_SPSS.sps` (remove the file name); the quotes or the final period are missing; the files are still inside the zip; OneDrive or iCloud has not finished downloading the folder. If H0 says a file cannot be found, check the path first. If the path itself contains an apostrophe (`'`), put the path in double quotes instead, for example `CD "/Users/you/Kim's files/LAB01_Student_Package".`

How every card works

 Yellow cells: you type here  Lavender cells: guidance

✓ **Check your work** box on each sheet turns to "✓ Checked"

Each card opens with a short recap of the session content it uses, with the slides and Toolkit pages to reread. Type each formula starting with  =, then press **Enter**. The cells are already formatted, so never multiply a percentage by 100.

Part A · Can we trust the order file?

[From slide 26](#) → work this card → [Back to slide 27](#)

· bring: Number of sentinel rows identified and confirmed scale of OrderTier.

WHAT THE SESSION COVERED · SECTION 1 DATA QUALITY, PROVENANCE, AND MEASUREMENT SCALES

Before you average anything, check two things: what each column *measures*, and which stored values are *not* measurements at all.

- **Measurement scales:** Stevens' four scales decide which arithmetic is allowed. **Nominal:** count and mode. **Ordinal:** rank and median. **Interval:** mean and difference. **Ratio:** mean and ratio.
- **Digits are not always numbers:** an order ID or a code stored as digits can still be a label. How a value is stored does not decide its scale; what it means does.
- **Grain:** know what one row represents before you count. In `Orders`, one row is one recorded order.
- **Sentinel codes:** placeholders such as -1 for age and 999 for latency mean *missing*. Count them and exclude them inside formulas. Never delete or overwrite them in the source.
- **Valid denominator:** $n_{\text{valid}} = N - \text{documented exclusions}$. Every later statistic divides by this number, not by all rows.

Why missingness matters: missing values are *MCAR* (completely at random), *MAR* (related to other recorded fields) or *MNAR* (related to the missing value itself). Dropping rows is only safe when the gaps are close to random, so note where the sentinel rows sit before you exclude them.

COURSE SLIDES

Slides 10-26

The Meaning of Measurement Scales in Business... 12 · Mathematical Admissibility by Measurement Scale 13 · SPSS and Excel Scale Validation Syntax 14 · What Can We Calculate? 15 · What Does One Row Represent? 16 · Excel Robust Data Screening and Missing Value... 17

CONCISE TOOLKIT

pp. 4-8

FULL TOOLKIT

pp. 4-24

Metro E-Commerce Logistics
p. 24

THE EQUATIONS YOU ARE BUILDING

N = rows in the order file · x_i = recorded latency of order i , in minutes · $\text{count}(\cdot)$ = number of rows that meet the condition

$$n_{-1} = \text{count}(\text{Age}_i = -1) = B7 \quad n_{999} = \text{count}(x_i = 999) = B8$$

THE DENOMINATOR EVERY LATER PART USES

$$n_{\text{valid}} = N - n_{999} - n_{\text{blank}}$$

FIRST, THE SETUP · ONCE PER SPSS SESSION

Run this **once each time you open SPSS**, before any part. On the `CD` line, replace `CHANGE_TO_YOUR_EXTRACTED_LAB_FOLDER` with the path of your **unzipped lab folder** (keep the single quotes and the final period). It is the folder that holds `LAB01_SPSS.sps` and the data files, **not** the `.sps` file itself: see [Find your folder path](#). Select from the first line down to the end of the H0 block and choose **Run > Selection**. Already ran it since you opened SPSS? Skip to the block for this part.

```
* Lecture connection: Observational Grain and Missingness Screening .
* Why: Import all 10,000 recorded orders with correct types and establish variable-specific
missing-value rules .
* How: GET DATA reads the exact CSV and declares identifier fields as strings .
* How: MISSING VALUES marks -1 only for age and 999 only for latency .
* How: FREQUENCIES on Hub provides an imported-case check .
* Read the result: H0 should show 10,000 imported cases while preserving every source row .
* Watch out: Do not edit the CSV or import identifiers as numbers .
* Run: Choose File > Open > Syntax and open the personalized LAB01 syntax .
* Run: Select the complete H0 block, including GET DATA .
* Run: Choose Run > Selection .
* Run: Locate H0 Import and Data Screen in the Viewer .
* LAB01 Metro E-Commerce Logistics.
* Update only the folder path below, then run H0 before later blocks.
CD 'CHANGE_TO_YOUR_EXTRACTED_LAB_FOLDER'. * ← change only this path
* Change an SPSS setting for this session.
```

```
SET DECIMAL=DOT.
```

```
* Name the output block so you can find it in the Viewer.
```

```
TITLE 'H0 Import and Data Screen'.
```

```
* Read the data file into SPSS (file type, name, and each variable's format).
```

```
GET DATA
```

```
  /TYPE=TXT
```

```
  /FILE='orders_story.csv'
```

```
  /ENCODING='UTF8'
```

```
  /DELCASE=LINE
```

```
  /DELIMITERS=", "
```

```
  /QUALIFIER='\"'
```

```
  /ARRANGEMENT=DELIMITED
```

```
  /FIRSTCASE=2
```

```
  /VARIABLES=
```

```
    OrderID A8
```

```

CustomerID A5
OrderTier F1.0
Hub A5
Age_Years F3.0
Latency_Minutes F8.3.
* Give the open dataset a name so later commands can point to it.
DATASET NAME OrdersData.
* Set each variable's measurement level (nominal, ordinal, scale).
VARIABLE LEVEL OrderID CustomerID Hub (NOMINAL) OrderTier (ORDINAL) Age_Years Latency_Minutes
(SCALE).
* Attach readable descriptions to variables (they appear in output tables).
VARIABLE LABELS
  OrderID 'Unique recorded order identifier'
  CustomerID 'Customer identifier; repetition does not imply duplicate orders'
  OrderTier 'Ordered service tier; median and ranks are admissible'
  Hub 'Regional fulfillment hub'
  Age_Years 'Recorded age in years; -1 is a sentinel'
  Latency_Minutes 'Fulfillment latency in minutes; 999 is a sentinel'.
* Declare which codes mean "missing" so they are left out of statistics.
MISSING VALUES Age_Years (-1) Latency_Minutes (999).
* Frequency table: how many cases fall in each category.
FREQUENCIES VARIABLES=Hub.
* Run the pending transformations now.
EXECUTE.

```

BEFORE THE TIMER · INSTRUCTOR DEMO

Follow along: in SPSS, run the **H1** block and leave **H1 Scale and Sentinel Audit** open.

SPSS BLOCK FOR THIS PART · ALREADY IN YOUR SYNTAX FILE

You do not type this: it is already in `LAB01_SPSS.sps`, the one syntax file you use all session. Once per SPSS session, run the `CD` line and the H0 block first (the setup code is above). Then press **Ctrl+F**, search `TITLE 'H1'`, select from the first comment line down to the last period, and choose **Run > Selection**. Green lines start with `*`: they are comments that explain the command below them, and SPSS ignores them. Every command ends with a period.

```

* Lecture connection: Measurement Scale Admissibility .
* Why: Inspect sentinel categories and confirm the assigned identifier, ordinal, nominal and
scale roles before calculating summaries .
* How: DISPLAY DICTIONARY shows variable roles and formats .
* How: FREQUENCIES includes user-missing sentinel categories and the OrderTier distribution .
* Read the result: Read the -1 and 999 rows as documented missing codes and confirm OrderTier is
ordinal .
* Watch out: Numeric storage does not make OrderTier a scale measurement .
* Run: Run H0 first in the same clean session .
* Run: Select the complete H1 block and choose Run > Selection .
* Run: Locate H1 Scale and Sentinel Audit .
* Run: Read the dictionary and frequency output .
TITLE 'H1 Scale and Sentinel Audit'.

```

```

DISPLAY DICTIONARY /VARIABLES=OrderID CustomerID OrderTier Hub Age_Years Latency_Minutes.
* Frequency table: how many cases fall in each category.
FREQUENCIES VARIABLES=OrderTier Age_Years Latency_Minutes
/MISSING=INCLUDE
/ORDER=ANALYSIS.

```

OPEN

Open **IN-CLASS_Lab Part A!A7** .

FILL IN THE YELLOW CELLS

#	Cell	Type this	In math	Check
1	B7	=COUNTIF(Orders[Age_Years], -1)	count($\text{Age}_i = -1$) age sentinels	42
2	B8	=COUNTIF(Orders[Latency_Minutes], 999)	count($x_i = 999$) latency sentinels	42
3	B9 B10 B14	B9: pick the best statement from the dropdown. B10: 2-4 sentences: state your choice, cite the relevant statistical rule or evidence, explain why it fits, and state one limitation. B14: caption the SPSS H1 Scale and Sentinel Audit output. 1-2 sentences: identify the Variable Information and relevant Frequency Tables you read, and describe the measurement-level and sentinel checks you made using your own output.		

✅ **Done when:** B7/B8 count the documented sentinels; B9 identifies an admissible scale statement. **Save.**

⚠️ **Most common mistake:** subtracting *both* sentinel counts from 10,000. Each code belongs to its own column. Latency results exclude only the 999 codes in Latency_Minutes ; the -1 codes affect age, not latency. $n_{\text{valid}} = 10,000 - 42 = 9,958$ not $10,000 - (42 + 42) = 9,916$

Stuck? Three quick fixes

- **#NAME? or #REF! error.** The data table must be named **Orders** with the headers **Age_Years** and **Latency_Minutes** . Don't rename the table or its columns.
- **B7 or B8 shows 0.** Check the column: -1 lives in **Age_Years** and 999 lives in **Latency_Minutes** . Type the criterion as a plain number, e.g. **COUNTIF(Orders[Age_Years], -1)** .
- **Tempted to delete the sentinel rows.** Don't. **Data** must keep all 10,000 rows. The later formulas skip the codes for you with **<>999** .

Why keep the sentinel rows instead of deleting them? (optional)

Deleting rows destroys the audit trail. Your VP, or a grader, must be able to see the codes and check your rule. Excluding them inside each formula keeps the source intact and makes the rule visible. It also lets you ask *why* they are missing: if the missing values cluster in one hub or one kind of order, the gaps are not random.

Part B · What does a typical order look like?

[From slide 45](#) → work this card → [Back to slide 46](#)

· bring: Calculated sample median and IQR for fulfillment duration.

WHAT THE SESSION COVERED · SECTION 2 DESCRIBING DISTRIBUTIONS: CENTER, SPREAD, AND SHAPE

Center and spread answer two different questions: where does a typical order sit, and how much do orders vary around it?

- **Mean:** the balance point, $\bar{x} = \frac{1}{n} \sum x_i$. Every value pulls on it, so a few very long deliveries move it.
- **Median:** the middle ranked value $\bar{x}$. Half the orders are faster and half are slower. Its 50% breakdown point means extreme values barely move it.
- **Sample standard deviation:** the typical distance from the mean, $s = \sqrt{\sum (x_i - \bar{x})^2 / (n - 1)}$. Squaring the distances makes it sensitive to the tail.
- **Interquartile range:** the width of the middle half, $IQR = Q_3 - Q_1$. Like the median, it resists extreme values.
- **Shape:** skewness tells you which side has the longer tail. Comparing the mean with the median is a quick first check. The 68-95-99.7 rule assumes a symmetric, normal shape, so don't apply it to skewed latency.

COURSE SLIDES

Slides 27–45

What Measures of Center
Mean 24 · Measures of Central
Tendency 25 · How Resistance
and Breakdown Points Operate
26 · Spread and Distribution
Shape 27 · Sample Variance
and Sample Standard
Deviation... 28 · Excel
Descriptive Dispersion and
Shape Formulas 29–30

CONCISE TOOLKIT

pp. 9–13

FULL TOOLKIT

pp. 25–47

Central Tendency and
Resistance p. 26 · Metro E-
Commerce Logistics p. 46

THE EQUATIONS YOU ARE BUILDING

x_i = valid latency of order i , in minutes · n = valid orders · Q_1, Q_3 = first and third quartiles

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \text{B7} \quad s = \frac{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}}{n-1} = \text{B9}$$

RESISTANT MEASURES

$\bar{x}$ = middle ranked x_i = B8 IQR = $Q_3 - Q_1$ = B10

WHERE QUARTILE.EXC LOOKS

Q_p sits at rank $p(n+1)$ in the sorted valid values

Just opened SPSS? Run the setup first (the **CD** line and H0; code in Part A).

BEFORE THE TIMER · INSTRUCTOR DEMO

Follow along: in SPSS, run the **H2** block and leave **H2 Center and Spread** open.

SPSS BLOCK FOR THIS PART · ALREADY IN YOUR SYNTAX FILE

You do not type this: it is already in **LAB01_SPSS.sps**, the one syntax file you use all session. Once per SPSS session, run the **CD** line and the H0 block first (setup code in Part A). Then press **Ctrl+F**, search

TITLE 'H2', select from the first comment line down to the last period, and choose **Run > Selection**.

Green lines start with *****: they are comments that explain the command below them, and SPSS ignores them. Every command ends with a period.

```
* Lecture connection: Central Tendency and Resistance .
* Why: Produce valid-latency center, sample spread and exclusive-method quartiles for comparison
with Excel .
* How: EXAMINE excludes the declared 999 user-missing values .
* How: HAVERAGE percentiles match the required exclusive position convention .
* How: Descriptives reports valid N, mean and sample standard deviation .
* Read the result: Read valid N, mean, median, SD, Q1 and Q3; calculate IQR as Q3 minus Q1 when
needed .
* Watch out: Do not infer the median from the mean or another table .
* Run: Select the complete H2 block after H0 and H1 .
* Run: Choose Run > Selection .
* Run: Locate H2 Center and Spread .
* Run: Read Descriptives and Percentiles, including the named median .
TITLE 'H2 Center and Spread'.
* Explore a variable: percentiles, spread and plots, optionally by group.
EXAMINE VARIABLES=Latency_Minutes
/PLOT=NONE
/STATISTICS=DESCRIPTIVES
/PERCENTILES(25,50,75) HAVERAGE
/MISSING=LISTWISE
/NOTOTAL.
```

OPEN

Open IN-CLASS_Lab Part B!A7 .

FILL IN THE YELLOW CELLS

#	Cell	Type this	In math	Check
1	B7	<code>=AVERAGE(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")))</code>	$\bar{x}$ mean, minutes	28.4
2	B8	<code>=MEDIAN(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")))</code>	$\bar{x}$ median, minutes	22.0
3	B9	<code>=STDEV.S(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")))</code>	s sample SD	13.5
4	B10	<code>=LET(v,FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")),QUARTILE.EXC(v,3)- QUARTILE.EXC(v,1))</code>	$Q_3 - Q_1$ exclusive quartiles	12.0
5	B12 B13 B17	B12: pick the best statement from the dropdown. B13: 2-4 sentences: state your choice, cite the relevant statistical rule or evidence, explain why it fits, and state one limitation. B17: caption the SPSS H2 Center and Spread output. 1-2 sentences: name the exact SPSS output, identify the statistic and population or filter you read, then state the cross-tool check you performed.		

✅ **Done when:** B7:B10 contain numeric minute results using the valid-latency rule. **Save.**

⚠️ **Most common mistake:** averaging the whole column. `AVERAGE(Orders[Latency_Minutes])` treats the 42 codes of 999 as real 999-minute deliveries. Every formula on this sheet must keep the `FILTER(...<>999...)` rule. $\frac{9,958 \times 28.4 + 42 \times 999}{10,000} \approx 32.5 \text{ min} \neq \bar{x} = 28.4 \text{ min}$

Stuck? Three quick fixes

- **Mean shows about 32.5.** The 999 codes are still in. Keep both filter conditions:
`(Orders[Latency_Minutes]<>999)*(Orders[Latency_Minutes]<>"")`.
- **#NAME? or #CALC! error.** `FILTER` and `LET` need Excel for Microsoft 365. Use the desktop or web version of Microsoft 365, not an older Excel.
- **B10 is negative.** You subtracted in the wrong order. The IQR is quartile 3 minus quartile 1:
`QUARTILE.EXC(v,3)-QUARTILE.EXC(v,1)`.

Why does a long tail move the mean but not the median? (optional)

$$\bar{x} = 22.0 \text{ min} \quad \bar{x} = 28.4 \text{ min} \quad \bar{x} - \bar{x} = 6.4 \text{ min}$$

The mean adds up every value, so a few very long deliveries pull it up. The median depends only on which order sits in the middle, so it stays put. To watch this happen, try Topic 2 (*Resistance*) in the optional practice. It adds the 999 codes back in, in the tool of your choice.

Part C · Which orders are unusually slow?

From slide 67 → work this card → Back to slide 68

· bring: Computed upper fence value and count of flagged delay incidents.

WHAT THE SESSION COVERED · SECTION 3 VISUAL EVIDENCE, HONEST DISPLAYS, AND DECLUTTERING

A **boxplot** draws the middle half of the data as a box and marks unusual values with **Tukey fences**. Honest charts let the reader see the real shape.

- **Tukey fences:** Lower = $Q_1 - 1.5IQR$ and Upper = $Q_3 + 1.5IQR$. Values beyond a fence are *flagged*.
- **Built from resistant pieces:** the fences use quartiles, so the extreme values they flag can't drag the fences outward the way they would drag a mean-plus-SD rule.
- **Honest histograms:** minutes are continuous, so the bars should touch with no gaps. Bar charts of amounts should start their value axis at zero.
- **Declutter:** remove non-data ink (3D effects, heavy gridlines, rainbow colors) and use one highlight color for the point that matters. Data-ink ratio = data ink/total ink.

COURSE SLIDES

Slides 47–67

Visual Choice and Distribution
Geometry 38 · Tukey Outlier
Fence Boundaries 39 · SPSS
and Excel Formulas for
Distribution Analysis 40–41 ·
Chart Geometry and Baselines
42 · Cognitive Load and Visual
Clutter 43 · Data-Ink Ratio
Formulation 44–45

CONCISE TOOLKIT

pp. 14–18

FULL TOOLKIT

pp. 48–69

Metro E-Commerce Logistics
p. 68

SPSS vs. Excel: the H3 boxplot draws its box from *Tukey hinges*, which can differ slightly from `QUARTILE.EXC`. Check B7 and B8 against the **HVERAGE Percentiles** table, not the plot. Use the plot only for its shape and the labeled extreme orders.

THE EQUATIONS YOU ARE BUILDING

Q_1, Q_3 = exclusive quartiles of valid latency · IQR = Part B cell B10 · 1.5 = Tukey's multiplier · $x_{(n)}$ = largest valid value

Upper = $Q_3 + 1.5\text{IQR}$ = B7 Lower = $Q_1 - 1.5\text{IQR}$ = B8

FLAG COUNT, VALID ORDERS ONLY

B9 = count($x_i > \text{Upper}$, $x_i \neq 999$)

Just opened SPSS? Run the setup first (the **CD** line and H0; code in Part A).

BEFORE THE TIMER · INSTRUCTOR DEMO

Follow along: in SPSS, run the **H3** block and leave **H3 Tukey and Boxplot Evidence** open.

SPSS BLOCK FOR THIS PART · ALREADY IN YOUR SYNTAX FILE

You do not type this: it is already in **LAB01_SPSS.sps**, the one syntax file you use all session. Once per SPSS session, run the **CD** line and the H0 block first (setup code in Part A). Then press **Ctrl+F**, search **TITLE 'H3'**, select from the first comment line down to the last period, and choose **Run > Selection**. Green lines start with *****: they are comments that explain the command below them, and SPSS ignores them. Every command ends with a period.

```
* Lecture connection: Visual Distribution Geometry and Tukey Fences .
* Why: Display the valid latency distribution and identify extreme recorded cases without
deleting them .
* How: EXAMINE uses OrderID as the case label .
* How: The boxplot and Extreme Values table use latency after declared missing values are
excluded .
* How: HAVERAGE percentile-table Q1/Q3 support the exclusive numerical check; the native plot
uses Tukey hinges .
* Read the result: Use HAVERAGE Q1/Q3 for the exclusive numerical check; Inspect the hinge-based
plot qualitatively; its near-boundary flags may differ from the Excel fence .
* Watch out: A plotted extreme value does not prove an error or a cause .
* Run: Select the complete H3 block and choose Run > Selection .
* Run: Locate H3 Tukey and Boxplot Evidence .
* Run: Read HAVERAGE Q1/Q3 in Percentiles for the numerical check; inspect the native plot
qualitatively and the Extreme Values table as ranked cases .
* Native boxplot: Tukey hinges for qualitative inspection only.
* For the Excel exclusive check, read HAVERAGE Q1 and Q3 in Percentiles.
* Do not require identical near-boundary case flags from the native plot.
TITLE 'H3 Tukey and Boxplot Evidence'.
* Explore a variable: percentiles, spread and plots, optionally by group.
EXAMINE VARIABLES=Latency_Minutes
/ID=OrderID
/PLOT=BOXPLOT
/STATISTICS=DESCRIPTIVES EXTREME(5)
/PERCENTILES(25,50,75) HAVERAGE
```

/MISSING=LISTWISE

/NOTOTAL.

OPEN

Open IN-CLASS_Lab Part C!A7 .

FILL IN THE YELLOW CELLS

#	Cell	Type this	In math	Check
1	B7	<code>=QUARTILE.EXC(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")),3)+1.5*'IN- CLASS_Lab Part B'!B10</code>	$Q_3 + 1.5\text{IQR}$ upper fence	49.0
2	B8	<code>=QUARTILE.EXC(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")),1)-1.5*'IN- CLASS_Lab Part B'!B10</code>	$Q_1 - 1.5\text{IQR}$ lower fence	1.0
3	B9	<code>=COUNTIFS(Orders[Latency_Minutes], " <>"&999,Orders[Latency_Minutes], " <>",Orders[Latency_Minutes], ">"&B7)</code>	$\text{count}(x_i > B7)$ whole records	829
4	B10	<code>=MAX(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")))</code>	$x_{(n)}$ largest	124.0
5	B11	<code>=LARGE(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)* (Orders[Latency_Minutes]<>"")),2)</code>	$x_{(n-1)}$ second largest	124.0
6	B13 B14 B18	B13: pick the best statement from the dropdown. B14: 2-4 sentences: describe the visible pattern, cite the relevant measure and units, explain what it means, and state what the chart cannot establish. B18: caption the SPSS H3 Tukey and Boxplot Evidence output. 1-2 sentences: name the exact SPSS output, identify the statistic and population or filter you read, then state the cross-tool check you performed.		

✓ Done when: B8<B7, B9 is a whole-record flag count, and B10>=B11. Save.

⚠ **Most common mistake:** adding $1.5 \times \text{IQR}$ to the *median* instead of to Q_3 . The fence starts at the edge of the box, not at its middle.

Upper = $Q_3 + 1.5\text{IQR} = 31 + 1.5(12) = 49.0$ not $22 + 1.5(12) = 40.0$

Stuck? Three quick fixes

- **#REF! in B7 or B8.** Both fences read the IQR from 'IN-CLASS_Lab Part B'!B10. Finish Part B first and don't rename that sheet.
- **B9 looks too high.** The 999 codes may be counted as slow orders. Keep all three COUNTIFS criteria: "<>"&999, "<>" and ">"&B7.
- **B10 and B11 are equal. Is that wrong?** No. LARGE(..., 2) returns the second value in sorted order, and two orders can share the same latency.

Why 1.5 times the IQR? (optional)

Tukey picked 1.5 as a practical rule of thumb. For normal data, the fences sit about 2.7 standard deviations from the mean, so fewer than 1% of values are flagged. Skewed data such as latency flag many more values on the long side. Try other multipliers in Topic 3 (*fence sensitivity*) in the optional practice.

Part D · Is Hub C really different?

[From slide 86](#) → work this card → [Back to slide 90](#)

· bring: Completed hub counts, medians and IQRs, with count reconciliation.

WHAT THE SESSION COVERED · SECTION 4 GROUPED SUMMARIES, COMPARATIVE DISTRIBUTIONS, AND LAB PRACTICE

To compare groups fairly, keep the data **stacked** (one row per order, one column for the group) and apply the *same* rules to every group.

- **Stacked layout:** `Orders` has one row per order, with `Hub` in one column and `Latency_Minutes` in another. That is what lets one formula, one PivotTable or one SPSS `BY Hub` summarize every hub.
- **Audit standard:** report each group's sample size, center and spread together, all from the same valid-latency filter and the same quartile rule.
- **Reconcile the counts:** the hub counts must add up: $n_A + n_B + n_C = n_{\text{all}}$. If they don't, a filter differs somewhere.
- **Pooled, not averaged:** the all-hubs median and IQR come from all valid orders pooled together, not from averaging the three hub values.
- **Causal boundary:** a group summary describes the group, not each order in it (the *ecological fallacy*). A difference between hubs is a description, not proof that a hub caused it.

COURSE SLIDES

Slides 68–86

Stacked Data: One Observation per Row 55 · Subgroup Sample Mean and Pooled Variance 57 · SPSS Means and Group Breakdown Syntax 58 · How a PivotTable Groups Records 59 · A Descriptive Audit 60 · Excel Dynamic Grouped Descriptive Summary and Audit... 61

CONCISE TOOLKIT

pp. 19–23

FULL TOOLKIT

pp. 70–94

Metro E-Commerce Logistics
p. 90

THE EQUATIONS YOU ARE BUILDING

$g = \text{hub (A, B or C)}$ · $n_g = \text{valid orders in hub } g$ · $\bar{x}_g = \text{hub median}$ · $Q_{1,g}, Q_{3,g} = \text{hub quartiles}$
 $n_g = \text{B7:B9}$ $\bar{x}_g = \text{C7:C9}$ $\text{IQR}_g = Q_{3,g} - Q_{1,g} = \text{F7:F9}$

RECONCILIATION CHECK

$n_A + n_B + n_C = \text{B7} + \text{B8} + \text{B9} = \text{B10}$

Just opened SPSS? Run the setup first (the **CD** line and H0; code in Part A).

BEFORE THE TIMER · INSTRUCTOR DEMO

Follow along: in SPSS, run the **H4** block and leave **H4 Hub Comparative Audit** open.

SPSS BLOCK FOR THIS PART · ALREADY IN YOUR SYNTAX FILE

You do not type this: it is already in **LAB01_SPSS.sps**, the one syntax file you use all session. Once per SPSS session, run the **CD** line and the H0 block first (setup code in Part A). Then press **Ctrl+F**, search **TITLE 'H4'**, select from the first comment line down to the last period, and choose **Run > Selection**. Green lines start with *****: they are comments that explain the command below them, and SPSS ignores them. Every command ends with a period.

```
* Lecture connection: Comparative Distribution Audits and Causal Boundaries .  
* Why: Compare valid count, center and spread across Hub A, Hub B and Hub C using one common rule .  
*  
* How: BY Hub creates separate summaries from the stacked Hub column .  
* How: Each group uses its own valid latency denominator .  
* How: HAVERAGE provides the required quartile convention .  
* Read the result: Compare each hub's valid N, median, Q1 and Q3 without assigning operational cause .  
* Watch out: Do not compare groups that use different missing-value filters .  
* Run: Select the complete H4 block and choose Run > Selection .  
* Run: Locate H4 Hub Comparative Audit .  
* Run: Read each hub's valid N and percentile table .  
* Run: Confirm the group counts reconcile .  
TITLE 'H4 Hub Comparative Audit'.  
* Explore a variable: percentiles, spread and plots, optionally by group.  
EXAMINE VARIABLES=Latency_Minutes BY Hub  
/PLOT=NONE  
/STATISTICS=DESCRIPTIVES  
/PERCENTILES(25,50,75) HAVERAGE  
/MISSING=LISTWISE.
```

OPEN

Open **IN-CLASS_Lab Part D!A7**.

FILL IN THE YELLOW CELLS

#	Cell	Type this	In math	Check
1	B7	<code>=COUNTIFS(Orders[Hub],\$A7,Orders[Latency_Minutes], "<>"&999,Orders[Latency_Minutes], "<>")</code> Fill to B9.	n_g valid count per hub	3,319, 3,319, 3,320
2	C7	<code>=MEDIAN(FILTER(Orders[Latency_Minutes], (Orders[Hub]=\$A7)*(Orders[Latency_Minutes]<>999)*(Orders[Latency_Minutes]<>"")))</code> Fill to C9.	$\bar{x}_g$ hub median	22.0, 22.0, 22.0
3	D7	<code>=QUARTILE.EXC(FILTER(Orders[Latency_Minutes], (Orders[Hub]=\$A7)*(Orders[Latency_Minutes]<>999)*(Orders[Latency_Minutes]<>"")),1)</code> Fill to D9.	$Q_{1,g}$ exclusive Q1	19.0, 19.0, 14.0
4	E7	<code>=QUARTILE.EXC(FILTER(Orders[Latency_Minutes], (Orders[Hub]=\$A7)*(Orders[Latency_Minutes]<>999)*(Orders[Latency_Minutes]<>"")),3)</code> Fill to E9.	$Q_{3,g}$ exclusive Q3	26.0, 26.0, 31.0
5	B10	<code>=COUNTIFS(Orders[Latency_Minutes], "<>"&999,Orders[Latency_Minutes], "<>")</code>	n_{all} must equal the sum	9,958
6	C10	<code>= 'IN-CLASS_Lab Part B'!B8</code>	$\bar{x}_{all}$ linked from Part B	22.0
7	D10	<code>=QUARTILE.EXC(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)*(Orders[Latency_Minutes]<>"")),1)</code>	$Q_{1,all}$	19.0
8	E10	<code>=QUARTILE.EXC(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)*(Orders[Latency_Minutes]<>"")),3)</code>	$Q_{3,all}$	31.0
9	F7	<code>=E7-D7</code> Fill to F10.	$Q_3 - Q_1$ IQR, rows 7 to 10	7.0, 7.0, 17.0, 12.0

#	Cell	Type this	In math	Check
		B12: One chart-supported sentence: state the comparison, name the measure, and cite the visible chart or table evidence without claiming causation.		
B12		B13: 2-4 sentences: describe the visible pattern, cite the relevant measure and units, explain what it means, and state what the chart cannot establish.		
10	B13			
B17		B17: caption the SPSS H4 Hub Comparative Audit output. 1-2 sentences: name the exact SPSS output, identify the statistic and population or filter you read, then state the cross-tool check you performed.		

✓ **Done when:** B7:F10 is complete; hub counts sum to the all-hubs count and each IQR equals Q3 minus Q1. **Save.**

⚠ **Most common mistake:** averaging the three hub IQRs to fill F10. The all-hubs IQR comes from all 9,958 valid orders pooled together, so it is not the average of the hub values.

$$\frac{7 + 7 + 17}{3} \approx 10.3 \neq \text{IQR}_{\text{all}} = 31 - 19 = 12.0$$

Stuck? Three quick fixes

- **Rows 8 and 9 repeat Hub A.** Lock the column only: `$A7` , not `$A$7` , so the row moves as you fill down.
- **Hub A shows Q1 = 20.5 and Q3 = 24.0.** You used `QUARTILE.INC` . Switch to `QUARTILE.EXC` , the rule SPSS `HAVERAGE` uses.
- **Hub counts don't add up to B10.** Every row must use the same two filters, `<>999` and not blank. Also check that the names in A7:A9 match `Data` exactly, e.g. `Hub A` .

Why store the data stacked? (optional)

If each hub had its own column (a *wide* layout), you would need a different formula for every hub and could not pivot or run `BY Hub` . With stacked data, one formula filled down handles every hub. Adding a Hub D next year means adding rows, not rebuilding the workbook.

Take-home · Should we penalize Hub C, and what should the SLA say?

🕒 60 min

At home → work this card → Submit

WHAT THE SESSION COVERED · SECTION 4 GROUPED SUMMARIES, COMPARATIVE DISTRIBUTIONS, AND LAB PRACTICE

The take-home reuses Parts A to D to answer the VP's two decisions. Each statistic answers a different SLA question:

- **Median:** the *typical* order. It resists the tail.
- **Mean:** the *average* minutes per order. Every minute counts, including the tail, so it can sit well above what a typical customer sees.
- **95th percentile:** the *upper-tail* threshold: 95% of valid orders are at or below P_{95} .
`PERCENTILE.EXC(v,0.95)` uses the same $p(n+1)$ rule as `QUARTILE.EXC`.
- **Share above a threshold:** a proportion of Hub C's own valid orders, so the denominator is n_C (B9), not the all-hubs count.
- **Causal boundary:** route mix, package mix or geography could also produce a hub difference. Name the evidence that would test a cause.

There are no dropdowns here. Write the evidence defense and the recommendation in your own words, with a unit and a denominator for every number.

COURSE SLIDES

Slides 68–86

Stacked Data: One Observation per Row 55 · Subgroup Sample Mean and Pooled Variance 57 · SPSS Means and Group Breakdown Syntax 58 · How a PivotTable Groups Records 59 · A Descriptive Audit 60 · Excel Dynamic Grouped Descriptive Summary and Audit... 61

CONCISE TOOLKIT

pp. 19–23

FULL TOOLKIT

pp. 70–94

Metro E-Commerce Logistics
p. 90

THE EQUATIONS YOU ARE BUILDING

$C = \text{Hub C} \cdot n_C = \text{Hub C valid orders} \cdot P_{95} = 95\text{th percentile of valid latency, all hubs} \cdot \bar{x}_C, \bar{x}_C = \text{Hub C mean and median}$

$P_{95} = \text{value at rank } 0.95(n + 1) \text{ of the sorted valid latencies} = \text{B8}$

MEAN-MINUS-MEDIAN GAP FOR HUB C

$\text{Gap}_C = \bar{x}_C - \bar{x}_C = \text{B10} - \text{B11} = \text{B13}$

SHARE OF HUB C ABOVE THE ALL-HUBS THRESHOLD

$\hat{p}_C = \frac{\text{count}(\text{Hub C: } x_i > P_{95})}{n_C} = \frac{\text{count}(\text{Hub C: } x_i > \text{B8})}{\text{B9}} = \text{B14}$

OPEN

Open `POST-CLASS_Part 1!A7`.

TRY IT FIRST · NEW FUNCTIONS

These functions are new in this part. Type each example in any empty cell and check you get the same result, then use the function below.

- `=PERCENTILE.EXC({10, 20, 30, 40, 50}, 0.5)` returns **30**: a percentile using the exclusive rule (p must stay away from 0 and 1).

This part is your independent practice: most formulas are not given. Each step names the in-class cell that used the same method, or gives a pattern to complete. The math column says what each cell must compute.

FILL IN THE YELLOW CELLS

#	Cell	Build it	In math
1	B7	<code>= 'IN-CLASS_Lab Part D'!___</code> Fill each ___. Combine the cells named in the math column.	n_{all} linked from Part D
2	B8	<code>=PERCENTILE.EXC(FILTER(Orders[Latency_Minutes], (Orders[Latency_Minutes]<>999)*(Orders[Latency_Minutes] <>"")),0.95)</code>	P_{95} all hubs, minutes
3	B9	Same method as Part C · B9 . Adapt it to this sheet.	n_C denominator for B14
4	B10	<code>=AVERAGE(___)</code> Fill each ___. Same idea as Part B · B7 .	$\bar{x}_C$

#	Cell	Build it	In math
5	B11	<code>=MEDIAN(____)</code> Fill each _____. Same idea as Part B · B8 .	x_C
6	B12	<code>=PERCENTILE.EXC(____, ____)</code> Fill each _____. Same idea as step 2 above · B8 .	$P_{95,C}$ Hub C only
7	B13	<code>=____-____</code> Fill each _____. Combine the cells named in the math column.	$x_C - \bar{x}_C$ minutes
8	B14	Same method as Part C · B9 . Adapt it to this sheet.	$\hat{p}_C$ percentage format

B16: 1-2 short paragraphs: identify evidence for typical service, variability, and upper-tail risk; include each value, unit, denominator, meaning, and one limitation.

B21: 1-2 short paragraphs: state the decision and recommended action, cite the key

B16 evidence with units and denominator, state one limitation, and name the next
B21 evidence needed.

9 B26 **B26:** 1-2 sentences: name the exact SPSS output, identify the statistic and population
B29 or filter you read, then state the cross-tool check you performed.

B33 **B29:** One line per memo claim: state the claim, then point to the exact workbook sheet and cell or named SPSS output that supports it.

B33: Either No AI used, or identify the tool, purpose, material adopted or revised, and the checks you performed.

✅ **Done when:** B7:B14 are complete, prose uses the student's own words, units and denominators are stated, and the recommendation does not claim causation. **Save.**

⚠️ **Most common mistake:** writing that Hub C *caused* the delays, or reporting the B14 share without its denominator. These are descriptive summaries of one extract. Say "of Hub C's valid orders" and name what else could explain a gap.

Stuck? Three quick fixes

- **B8 or B12 shows a percentage.** A percentile is measured in minutes. Format B8 and B12 as Number; only B14 uses Percentage.
- **#NUM! in B8 or B12.** `PERCENTILE.EXC` needs k strictly between 0 and 1. Type `0.95`, not `95`.
- **B14 looks wrong.** It divides by B9, the Hub C valid count. Check that B9 is filled and that the criterion is `">"&B8`.

Optional stretch: how sensitive is the guardrail? (ungraded)

In a *copy* of your finished workbook, change 0.95 in B8 to 0.90, then to 0.99. Does the Hub C share in B14 move a lot or a little? Write one sentence for yourself about how much the threshold choice matters. Don't submit the copy.

Submit

Brightspace · LAB01

What to hand in, and how it's graded

Due Tue, Sep 22 · 11:59 PM ET

YOUR EXACT FILENAMES

Upload	Filename	Must contain
Workbook	LAB01_FirstName_ LastName_NetID.xlsx	Every sheet: formulas, interpretations, SPSS captions, evidence map, AI disclosure.
Memo	LAB01_FirstName_ LastName_NetID.pdf	One page, using the template below. A .docx is also accepted; upload one format.
Keep, don't upload	LAB01_FirstName_ LastName_NetID.sps LAB01_FirstName_ LastName_NetID.spv	Your SPSS syntax and output. Upload only if Brightspace asks.

BEFORE YOU SUBMIT: TICK EVERY BOX 0 of 35 done

Files and names

- ☐ Workbook saved as LAB01_FirstName_
LastName_NetID.xlsx
- ☐ Memo exported as LAB01_FirstName_
LastName_NetID.pdf (or .docx ; upload
one format only)
- ☐ SPSS files kept as LAB01_FirstName_
LastName_NetID.sps and LAB01_
FirstName_LastName_NetID.spv (upload
only if Brightspace asks)
- ☐ NetID is the part before @nyu.edu (like
jd1234), not your N-number or full email.
Underscores between fields, no spaces.

Part A · IN-CLASS_Lab Part A

- ☐ B7/B8 count the documented sentinels;
B9 identifies an admissible scale
statement.
- ☐ B9 statement chosen
- ☐ B10 written: Reason
- ☐ B14 captions SPSS **H1 Scale and Sentinel
Audit**

Part B · IN-CLASS_Lab Part B

- ☐ B7:B10 contain numeric minute results using the valid-latency rule.
- ☐ B12 statement chosen
- ☐ B13 written: Reason
- ☐ B17 captions SPSS **H2 Center and Spread**

Part C · IN-CLASS_Lab Part C

- ☐ B8<B7, B9 is a whole-record flag count, and B10>=B11.
- ☐ B13 statement chosen
- ☐ B14 written: Reason and chart reading
- ☐ B18 captions SPSS **H3 Tukey and Boxplot Evidence**

Part D · IN-CLASS_Lab Part D

- ☐ B7:F10 is complete; hub counts sum to the all-hubs count and each IQR equals Q3 minus Q1.
- ☐ B12 written: Take-home: write your own chart-supported claim
- ☐ B13 written: Reason and chart reading
- ☐ B17 captions SPSS **H4 Hub Comparative Audit**

Take-home · POST-CLASS_Part 1

- ☐ B7:B14 are complete, prose uses the student's own words, units and denominators are stated, and the recommendation does not claim causation.
- ☐ B16 written: Evidence defense
- ☐ B21 written: Workbook recommendation source
- ☐ B26 written: SPSS H5 evidence caption
- ☐ B29 written: Memo evidence map (planning only)
- ☐ B33 written: AI-use or non-use disclosure

Whole workbook

- ☐ Every *Check your work* box reads "✓ Checked"
- ☐ No #NAME? , #VALUE! , #REF! , #NUM! or #DIV/0! anywhere
- ☐ Data sheets unchanged; formulas, not pasted values

Memo

- ☐ One page; To, From, Date and Subject filled in; leads with the recommendation
- ☐ Every number has a unit, a denominator and a source cell or SPSS output
- ☐ Separates association from cause, and names what would reverse the recommendation
- ☐ AI disclosure included; every [bracketed prompt] deleted

Upload

- ☐ Both files uploaded to the Brightspace LAB01 assignment by **Tuesday, September 22, 2026, at 11:59 PM ET**
- ☐ Reopened the uploaded files from Brightspace to confirm they open and are the final versions

GRADING MATRIX · 500 POINTS

Dimension	Points	Where it shows up in LAB01
Decision frame, KPI/guardrail, and stakeholder relevance	50 	Memo answers the penalty and SLA questions first; names the typical-service measure and the tail guardrail you chose, and why they matter to the VP.
Data provenance, grain, quality, and preparation	65 	10,000 rows unchanged; -1 and 999 counted (Part A B7:B8) and excluded, not deleted; valid denominator stated; SPSS H0 import checked.
Method fit and correct required Excel/SPSS implementation	115 	Formulas in Parts A-D and Take Home B7:B14 use the valid-latency filter and QUARTILE.EXC ; SPSS H1-H5 run and captioned.
Assumptions and diagnostic evidence	75 	Scale choice explained (Part A B9:B10); mean vs. median and Tukey fences read as evidence about shape (Parts B-C, SPSS H2-H3).
Uncertainty, effect size, and practical interpretation	70 	Hub differences in minutes (medians, IQRs, B13) and the B14 share with its Hub C denominator; no causal claim.
Independent validation, sensitivity, and reasonableness checks	55 	SPSS captions agree with Excel; hub counts reconcile to B10; every <i>Check your work</i> box reads "✓ Checked".
Recommendation, accessible evidence, and communication	45 	Memo leads with the recommendation; every number has a unit and denominator; the B29 evidence map traces each claim.
Reproducibility, sources, privacy, and AI disclosure/validation	25 	Correct filenames; formulas, not pasted values; .sps and .spv kept; AI disclosure in B33 and the memo.
Total	500	The rubric applies to the whole submission.

To: VP of Supply Chain Logistics, Metro E-Commerce Logistics
From: [Your name], Lead Operations Analytics Consultant
Date: [Submission date]
Subject: [Decision-focused subject, e.g. "Recommendation on Hub C penalties and the fulfillment SLA threshold"]

1. Recommendation 2–4 sentences

[Answer both decisions first. Penalty: do the records support penalizing Hub C now? SLA: which measure should set the typical-service target, and which should set the tail guardrail? Give the main business reason.]

2. Key evidence 4–6 rows

Finding	Value and unit	Denominator	Source
<i>[e.g. typical latency, all hubs]</i>	<i>[value, min]</i>	<i>[n = valid orders]</i>	<i>[Part B B8; SPSS H2]</i>
<i>[spread by hub]</i>	<i>[IQR, min]</i>	<i>[n per hub]</i>	<i>[Part D F7:F9; SPSS H4]</i>
<i>[upper-tail threshold]</i>	<i>[value, min]</i>	<i>[n]</i>	<i>[Take Home cell; SPSS H5]</i>
<i>[Hub C share above the threshold]</i>	<i>[%]</i>	<i>[n = Hub C valid orders]</i>	<i>[Take Home cell]</i>

3. Data and scope 1–2 sentences

[How many records you started with, the sentinel rule (-1 and 999 excluded, not deleted), the valid denominator, and what population and period the evidence describes.]

4. Uncertainty and limits 2–3 sentences

[What the evidence can't show: these are descriptive summaries, not proof that a carrier caused delays. Name at least one alternative explanation, such as route or package mix, and say whether the missing values look random.]

5. What would change my recommendation 1–2 sentences

[A specific condition that would reverse your advice, and the next evidence you would collect (for example, route-level or time-period data).]

AI disclosure: [*"No AI used." or the tool, task, what you kept, what you changed, and how you checked it.*]

⚠ Delete every [bracketed prompt] before submitting. Cite the key numbers; don't paste the evidence map into the memo.

Optional practice (not graded)

Everything here is optional and ungraded. Each topic asks one question and shows it in every tool that can answer it, in the order Excel, SPSS, Python and R, so pick the tool you want to practice. The **Reproduce** topic redoes the in-class Parts A-D so you can check your numbers from a second tool; it deliberately stops before the take-home. The **Extend** topics go one step beyond the lab. Put any code file in the same folder as `orders_story.csv`, and do Excel work in a *copy* of your workbook. Start after your workbook and memo are done.

Companion notebooks in your package:

- `LAB01_Python_Exploratory.ipynb`
- `LAB01_R_Exploratory.ipynb`

Topic 1 · Reproduce the in-class parts Reproduce

Same question in Excel → SPSS → Python → R.

Excel Parts A, B, C, D

No new formulas. Reopen your finished workbook and work through the sheets `IN-CLASS_Lab Part A` to `IN-CLASS_Lab Part D`. **Step 1.** Read column C next to each answer cell. It should say *Result matches*. **Step 2.** Scroll to the **Check your work** box under each table. Every line should read "✓ Checked". **Step 3.** If a cell says *Check the result*, click it and read the formula bar. Most slips are a missing `<>999` filter or `QUARTILE.INC` in place of `QUARTILE.EXC`. **Step 4.** On Part D, confirm that $B7 + B8 + B9$ equals B10, the 9,958 valid orders.

SPSS Parts A, B, C, D

No new syntax. Open LAB01_SPSS.sps , run H0, then run H1 to H4 one block at a time and compare each output with your workbook. **H1:** the -1 row for Age_Years and the 999 row for Latency_Minutes should each show 42 records (Part A B7:B8). **H2:** valid N, mean, median and SD match Part B B7:B9. Take Q3 minus Q1 from the **HAVERAGE** Percentiles table for B10. **H3:** $Q3 + 1.5 \times \text{IQR}$ from that table gives the 49.0-minute fence in Part C B7. Read the boxplot for shape only, since its hinges can differ slightly. **H4:** each hub's valid N, median, Q1 and Q3 match Part D rows 7 to 9. Stop at H4; H5 belongs to the take-home.

Counts the sentinel codes, then rebuilds every Part B, C and D value from the valid orders. Your printed numbers should match the workbook checks exactly, for example 9,958 valid orders, a 22.0-minute median, a 49.0-minute upper fence and 829 flagged records. `method="weibull"` is numpy's name for the `QUARTILE.EXC` rule. Runs in Colab or any Python with pandas and numpy.

```
# LAB01 Parts A-D in Python (optional, not graded).
# Put this file next to orders_story.csv.
import numpy as np
import pandas as pd

df = pd.read_csv("orders_story.csv", dtype={"OrderID": str, "CustomerID": str, "Hub": str})
print(f"Rows imported = {len(df):,}")    # should be 10,000; never edit the CSV

def q_exc(x, p):
    """Excel QUARTILE.EXC / PERCENTILE.EXC: the (n + 1) * p position rule."""
    return np.percentile(x, 100 * p, method="weibull")

# ---- Part A: sentinel audit ----
print(f"Age sentinels (-1)      = {(df['Age_Years'] == -1).sum()}")
print(f"Latency sentinels (999) = {(df['Latency_Minutes'] == 999).sum()}")
# Read the OrderTier codes and decide the scale yourself.
print(df["OrderTier"].value_counts().sort_index())

# ---- Part B: center and spread of VALID latency ----
valid = df[(df["Latency_Minutes"] != 999) & df["Latency_Minutes"].notna()]
lat = valid["Latency_Minutes"]
q1, q3 = q_exc(lat, 0.25), q_exc(lat, 0.75)
print(f"Valid latency records   = {len(lat):,}")
print(f"Mean latency              = {lat.mean():.1f} min")
print(f"Median latency             = {lat.median():.1f} min")
print(f"Sample SD                  = {lat.std(ddof=1):.1f} min")    # ddof=1 matches STDEV.S
print(f"IQR (Q3 - Q1)            = {q3 - q1:.1f} min")

# ---- Part C: Tukey fences ----
iqr = q3 - q1
upper, lower = q3 + 1.5 * iqr, q1 - 1.5 * iqr
print(f"Upper fence              = {upper:.1f} min")
print(f"Lower fence                = {lower:.1f} min")
print(f"Records above upper        = {(lat > upper).sum():,}")
top_two = lat.nlargest(2).to_numpy()
print(f"Largest valid latency     = {top_two[0]:.1f} min")
print(f"Second largest              = {top_two[1]:.1f} min")

# ---- Part D: grouped summary by hub (same valid-latency rule for every row) ----
def summary(x):
    a, b = q_exc(x, 0.25), q_exc(x, 0.75)
    return pd.Series({"Count": len(x), "Median": x.median(),
```

```
"Q1": a, "Q3": b, "IQR": b - a})
```

```
table = valid.groupby("Hub")["Latency_Minutes"].apply(summary).unstack()
table.loc["All Hubs"] = summary(lat)
print(table.to_string(float_format=lambda v: f"{v:,.1f}"))
hub_total = int(table.loc[["Hub A", "Hub B", "Hub C"], "Count"].sum())
print("Hub counts add up:", hub_total == len(lat))
```

R Parts A, B, C, D

The same checks in base R. `quantile(type = 6)` matches `QUARTILE.EXC` and SPSS `HAVERAGE`, and `sd()` matches `STDEV.S`.

```
# LAB01 Parts A-D in R (optional, not graded). Base R only.
# Put this file next to orders_story.csv.
df <- read.csv("orders_story.csv",
               colClasses = c(OrderID = "character", CustomerID = "character", Hub = "character"))
cat("Rows imported =", nrow(df), "\n") # should be 10000; never edit the CSV
# type 6 matches Excel QUARTILE.EXC and SPSS HAVERAGE
q_exc <- function(x, p) unname(quantile(x, p, type = 6))
f1 <- function(x) sprintf("%.1f", x)

# ---- Part A: sentinel audit ----
cat("Age sentinels (-1)      =", sum(df$Age_Years == -1), "\n")
cat("Latency sentinels (999) =", sum(df$Latency_Minutes == 999), "\n")
# Read the OrderTier codes and decide the scale yourself.
print(table(OrderTier = df$OrderTier))

# ---- Part B: center and spread of VALID latency ----
valid <- df[!is.na(df$Latency_Minutes) & df$Latency_Minutes != 999, ]
lat <- valid$Latency_Minutes
q1 <- q_exc(lat, 0.25); q3 <- q_exc(lat, 0.75)
cat("Valid latency records   =", length(lat), "\n")
cat("Mean latency           =", f1(mean(lat)), "min\n")
cat("Median latency         =", f1(median(lat)), "min\n")
cat("Sample SD              =", f1(sd(lat)), "min\n") # sd() divides by n - 1, like STDEV.S
cat("IQR (Q3 - Q1)          =", f1(q3 - q1), "min\n")

# ---- Part C: Tukey fences ----
iqr <- q3 - q1
upper <- q3 + 1.5 * iqr; lower <- q1 - 1.5 * iqr
top_two <- sort(lat, decreasing = TRUE)[1:2]
cat("Upper fence            =", f1(upper), "min\n")
cat("Lower fence           =", f1(lower), "min\n")
cat("Records above upper    =", sum(lat > upper), "\n")
cat("Largest valid latency   =", f1(top_two[1]), "min\n")
cat("Second largest         =", f1(top_two[2]), "min\n")

# ---- Part D: grouped summary by hub (same valid-latency rule for every row) ----
summ <- function(x) c(Count = length(x), Median = median(x),
                     Q1 = q_exc(x, 0.25), Q3 = q_exc(x, 0.75))
tab <- t(sapply(split(lat, valid$Hub), summ))
tab <- rbind(tab, "All Hubs" = summ(lat))
tab <- cbind(tab, IQR = tab[, "Q3"] - tab[, "Q1"])
```

```
print(round(tab, 1))
cat("Hub counts add up:", sum(tab[c("Hub A", "Hub B", "Hub C"), "Count"]) == length(lat), "\n")
```

Topic 2 · Resistance: what 999 codes do to the mean and median Extend

Same question in Excel → SPSS → Python → R.

Excel Parts A, B

Step 1. In a *copy* of your workbook, type `=AVERAGE(Orders[Latency_Minutes])` and `=MEDIAN(Orders[Latency_Minutes])` in two empty cells on the Part B sheet. These skip the filter, so the 42 codes of 999 count as real deliveries. **Step 2.** Compare them with B7 and B8. Which one moved, and by how much? **Step 3.** Check the mean by hand: $(9,958 \times B7 + 42 \times 999)/10,000$.

SPSS Parts A, B

One `EXAMINE` with `/MISSING=LISTWISE INCLUDE`, which treats the declared 999 codes as real minutes. Put its Descriptives table next to your H2 output. The median should stay at 22.0 minutes. What happened to the mean and the SD?

New code: paste it at the end of `LAB01_SPSS.sps`, make sure the setup has run this session, then select only the pasted lines and choose **Run > Selection**.

```
* Extension: what the 42 codes of 999 do to the mean and the median .
* Why: see which summary resists bad codes and which one does not .
* How: /MISSING=LISTWISE INCLUDE tells EXAMINE to treat user-missing 999 as real minutes .
* Read the result: compare the mean, median and SD with your H2 output .
* Watch out: this is a what-if; never report these numbers as the valid summary .
* Run: Run H0 first so OrdersData is open, then select this block and choose Run > Selection .
DATASET ACTIVATE OrdersData.
TITLE 'Extension Sentinels Left In'.
EXAMINE VARIABLES=Latency_Minutes
/PLOT=NONE
/STATISTICS=DESCRIPTIVES
/PERCENTILES(25,50,75) HAVERAGE
/MISSING=LISTWISE INCLUDE
/NOTOTAL.
```

Pretend some 999 codes slipped past your screen. Add 10, 42, 100 and more of them, then watch the mean and the standard deviation climb while the median stays at 22.0 minutes. The chart is saved as `resistance.png`.

```
# Extension: resistance in action (Part B).
# Add extreme delays and watch the mean and the median.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("orders_story.csv", dtype={"OrderID": str, "CustomerID": str, "Hub": str})
keep = (df["Latency_Minutes"] != 999) & df["Latency_Minutes"].notna()
lat = df.loc[keep, "Latency_Minutes"].to_numpy()

rows = []
for extra in [0, 10, 42, 100, 500, 1000]:
    x = np.append(lat, np.full(extra, 999.0)) # pretend 'extra' codes slipped in
    rows.append({"added 999s": extra, "mean": x.mean(),
                "median": np.median(x), "sample SD": x.std(ddof=1)})
result = pd.DataFrame(rows)
print(result.to_string(index=False, float_format=lambda v: f"{v:,.1f}"))

plt.plot(result["added 999s"], result["mean"], marker="o", label="Mean")
plt.plot(result["added 999s"], result["median"], marker="o", label="Median")
plt.xlabel("Number of 999 codes left in the data")
plt.ylabel("Minutes")
plt.title("The median resists bad codes; the mean does not")
plt.legend()
plt.savefig("resistance.png", dpi=120, bbox_inches="tight")
plt.show()
print("Saved resistance.png. The 42 real sentinels are the row with 42 added.")
```

R Parts B

The same sweep in base R, with the chart saved as `resistance_r.png`. The row with 0 added should show the Part B mean and median.

```
# Extension: resistance in action. Add extreme delays and watch the mean and median (Part B)
df <- read.csv("orders_story.csv",
               colClasses = c(OrderID = "character", CustomerID = "character", Hub = "character"))
lat <- df$Latency_Minutes[!is.na(df$Latency_Minutes) & df$Latency_Minutes != 999]
res <- data.frame()
for (extra in c(0, 10, 42, 100, 500, 1000)) {
  x <- c(lat, rep(999, extra)) # pretend 'extra' sentinel codes slipped in
  res <- rbind(res, data.frame(added = extra, mean = mean(x), median = median(x), sd = sd(x)))
}
print(round(res, 1))
png("resistance_r.png", width = 700, height = 450)
plot(res$added, res$mean, type = "b", pch = 19, ylim = c(0, max(res$mean)),
     xlab = "Number of 999 codes left in the data", ylab = "Minutes",
     main = "The median resists bad codes; the mean does not")
lines(res$added, res$median, type = "b", pch = 1, lty = 2)
legend("topleft", legend = c("Mean", "Median"), pch = c(19, 1), lty = c(1, 2))
dev.off()
cat("Saved resistance_r.png. The 42 real sentinels are the row with 42 added.\n")
```

Topic 3 · How sensitive are the Tukey fences? Extend

Same question in Excel → SPSS → Python → R.

Excel Parts C

In a *copy* of your workbook, add a new sheet. **Step 1.** Type 1.5, 2 and 3 in A2:A4. These are the multipliers k . **Step 2.** In B2 type `= 'IN-CLASS_Lab Part D'!$E$10+A2*'IN-CLASS_Lab Part D'!$F$10` and fill down to B4. Part D E10 is the all-hubs Q3 and F10 is the all-hubs IQR. **Step 3.** In C2 type `=COUNTIFS(Orders[Latency_Minutes], "<>999", Orders[Latency_Minutes], ">"&B2)` and fill down. The `<>999` criterion keeps the sentinel codes out. **Step 4.** Row 2 should match Part C: 49.0 minutes in B2 and 829 records in C2. How fast does the count fall as k grows?

Builds one flag per multiplier from the H2 quartiles, then counts each flag. The 999 codes are user-missing, so they get no flag. Which multiplier would you defend to the VP, and why?

New code: paste it at the end of `LAB01_SPSS.sps`, make sure the setup has run this session, then select only the pasted lines and choose **Run > Selection**.

** Extension: how many orders are flagged if the Tukey multiplier changes .*
** Why: a flag count depends on the rule; see how fast it falls as the fence widens .*
** How: 31 is the H2 HAVERAGE Q3 and 12 is the IQR (31 minus 19) .*
** How: 999 is user-missing, so those rows get no flag in any column .*
** Read the result: the value 1 row of Flag_k15 should match Part C B9 .*
** Run: Run H0 first, then select this block and choose Run > Selection .*

`DATASET ACTIVATE OrdersData.`

`TITLE 'Extension Fence Sensitivity'.`

`COMPUTE Flag_k15=(Latency_Minutes > 31 + 1.5*12).`

`COMPUTE Flag_k2=(Latency_Minutes > 31 + 2*12).`

`COMPUTE Flag_k3=(Latency_Minutes > 31 + 3*12).`

`VARIABLE LABELS Flag_k15 'Above Q3 + 1.5 IQR'`

`Flag_k2 'Above Q3 + 2 IQR'`

`Flag_k3 'Above Q3 + 3 IQR'.`

`EXECUTE.`

`FREQUENCIES VARIABLES=Flag_k15 Flag_k2 Flag_k3.`

Prints the fence and the flag count for $k = 1.5, 2$ and 3 , then charts the count for k from 1 to 3 as `fence_sensitivity.png`. The count axis starts at zero.

```
# Extension: how many orders are flagged if the Tukey multiplier changes? (Part C)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("orders_story.csv", dtype={"OrderID": str, "CustomerID": str, "Hub": str})
valid = df[(df["Latency_Minutes"] != 999) & df["Latency_Minutes"].notna()]
lat = valid["Latency_Minutes"]
q1, q3 = np.percentile(lat, [25, 75], method="weibull")      # QUARTILE.EXC
iqr = q3 - q1

rows = []
for k in [1.5, 2, 3]:
    fence = q3 + k * iqr
    rows.append({"k": k, "upper fence": fence, "flagged": int((lat > fence).sum())})
print(pd.DataFrame(rows).to_string(index=False, float_format=lambda v: f"{v:.1f}"))

# A finer sweep for the chart: how fast does the flag count fall as k grows?
ks = np.arange(1.0, 3.01, 0.25)
counts = [int((lat > q3 + k * iqr).sum()) for k in ks]
plt.plot(ks, counts, marker="o", color="black")
plt.axvline(1.5, linestyle="--", color="gray", label="Tukey's 1.5")
plt.xlabel("Multiplier k in Q3 + k * IQR")
plt.ylabel("Valid orders flagged")
plt.ylim(bottom=0)
plt.legend()
plt.savefig("fence_sensitivity.png", dpi=120, bbox_inches="tight")
print("Saved fence_sensitivity.png. At k = 1.5 you should see the Part C count.")
```

R Parts C

Recomputes the upper fence with multipliers 1.5, 2 and 3, and counts the flagged orders by hub each time. Which flags survive a stricter rule?

```
# Extension: how many orders are flagged if the Tukey multiplier changes? (Part C)
df <- read.csv("orders_story.csv",
               colClasses = c(OrderID = "character", CustomerID = "character", Hub = "character"))
valid <- df[!is.na(df$Latency_Minutes) & df$Latency_Minutes != 999, ]
q <- unname(quantile(valid$Latency_Minutes, c(0.25, 0.75), type = 6)) # QUARTILE.EXC
iqr <- q[2] - q[1]
for (k in c(1.5, 2, 3)) {
  fence <- q[2] + k * iqr
  flagged <- valid$Latency_Minutes > fence
  cat(sprintf("\nk = %.1f: upper fence = %.1f min, flagged = %d of %d valid orders\n",
             k, fence, sum(flagged), nrow(valid)))
  print(table(Hub = factor(valid$Hub[flagged], levels = sort(unique(valid$Hub)))))
}
# A wider fence flags fewer orders. Which hubs still have flags at k = 3?
```

Topic 4 · Histograms and boxplots by hub Extend

Same question in Excel → SPSS → Python → R.

Excel Parts C, D

This version draws the histograms; the boxplots are quicker in SPSS, Python or R. In a *copy* of your workbook, add a new sheet. **Step 1.** Type the bin tops 10, 20, ..., 130 in A2:A14, and Hub A, Hub B and Hub C in B1:D1. **Step 2.** In B2 type `=FREQUENCY(FILTER(Orders[Latency_Minutes], (Orders[Hub]=B$1)*(Orders[Latency_Minutes]<>999)), $A$2:$A$14)` and copy it to C2 and D2. Each column spills one count per bin. **Step 3.** Select B2:B14, choose **Insert > Column Chart**, and use **Select Data** to set A2:A14 as the axis labels. In **Format Data Series**, set **Gap Width** to 0% so the bars touch. **Step 4.** Repeat for Hub C and give both charts the same vertical axis maximum. Excel counts a value that equals a bin top in that bin, so a few counts can differ from Python and R.

SPSS Parts C, D

Adds a histogram and a boxplot for each hub to the Viewer. Compare where each hub's box sits and how far its right tail reaches.

New code: paste it at the end of `LAB01_SPSS.sps`, make sure the setup has run this session, then select only the pasted lines and choose **Run > Selection**.

```
* Extension: histograms and boxplots for each hub on the same scale .  
* Why: see whether the hubs differ in the middle, in the tail, or both .  
* How: EXAMINE BY Hub uses the declared missing values, so 999 is excluded .  
* Run: Run H0 first so OrdersData is open, then select this block and choose Run > Selection .  
DATASET ACTIVATE OrdersData.  
TITLE 'Extension Hub Distribution Shapes'.  
EXAMINE VARIABLES=Latency_Minutes BY Hub  
  /PLOT=BOXPLOT HISTOGRAM  
  /STATISTICS=NONE  
  /MISSING=LISTWISE  
  /NOTOTAL.
```

Draws one histogram per hub with the same 10-minute bins and axes, prints the bin counts, and adds side-by-side boxplots. The bars touch because minutes are continuous. Do the hubs differ in the middle, in the tail, or both?

```
# Extension: histograms and boxplots by hub on one shared scale (Parts C and D).
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("orders_story.csv", dtype={"OrderID": str, "CustomerID": str, "Hub": str})
valid = df[(df["Latency_Minutes"] != 999) & df["Latency_Minutes"].notna()]
bins = np.arange(0, 131, 10) # same 10-minute bins for every hub

# Count table first: one row per 10-minute bin, one column per hub.
binned = pd.cut(valid["Latency_Minutes"], bins=bins, right=False)
print(pd.crosstab(valid["Hub"], binned, dropna=False).T.to_string())

fig, axes = plt.subplots(3, 1, figsize=(7, 7), sharex=True, sharey=True)
for ax, (hub, group) in zip(axes, valid.groupby("Hub")):
    # No gaps: the bins touch because minutes are continuous.
    ax.hist(group["Latency_Minutes"], bins=bins, color="gray", edgecolor="white")
    ax.axvline(group["Latency_Minutes"].median(), color="black", linestyle="--", label="median")
    ax.set_title(f"{hub} (n = {len(group):,} valid orders)", loc="left")
    ax.legend()
axes[-1].set_xlabel("Fulfillment latency (minutes)")
fig.suptitle("Same center, different tails?")
fig.savefig("hub_histograms.png", dpi=120, bbox_inches="tight")

# Boxplots on the same minute scale; the box edges use numpy's default quartiles.
hubs = sorted(valid["Hub"].unique())
fig2, ax2 = plt.subplots(figsize=(6, 4))
ax2.boxplot([valid.loc[valid["Hub"] == h, "Latency_Minutes"] for h in hubs], whis=1.5)
ax2.set_xticks(range(1, len(hubs) + 1))
ax2.set_xticklabels(hubs)
ax2.set_ylabel("Fulfillment latency (minutes)")
fig2.savefig("hub_boxplots.png", dpi=120, bbox_inches="tight")
print("Saved hub_histograms.png and hub_boxplots.png.")
print("Compare the right tails, not only the medians.")
```

R Parts C, D

The same histograms in base R, plus a boxplot for each hub with the all-hubs upper fence as a dashed line. It prints the five numbers behind each box. R's box edges use Tukey hinges, so they can differ slightly from your `QUARTILE.EXC` values.

```
# Extension: histograms and boxplots of valid latency by hub (Parts C and D)
df <- read.csv("orders_story.csv",
               colClasses = c(OrderBy = "character", CustomerID = "character", Hub = "character"))
valid <- df[!is.na(df$Latency_Minutes) & df$Latency_Minutes != 999, ]
hubs <- sort(unique(valid$Hub))
brks <- seq(0, 130, by = 10) # same 10-minute bins for every hub
counts <- sapply(hubs, function(h)
  table(cut(valid$Latency_Minutes[valid$Hub == h], brks, right = FALSE)))
print(counts) # one row per bin, one column per hub

png("hub_histograms_r.png", width = 700, height = 700)
op <- par(mfrow = c(3, 1), mar = c(4, 4, 2, 1))
for (h in hubs) {
  x <- valid$Latency_Minutes[valid$Hub == h]
  hist(x, breaks = brks, right = FALSE, ylim = c(0, max(counts)),
       col = "gray", border = "white", xlab = "Fulfillment latency (minutes)",
       main = paste0(h, " (n = ", length(x), " valid orders)"))
  abline(v = median(x), lty = 2)
}
par(op)
dev.off()

q <- unname(quantile(valid$Latency_Minutes, c(0.25, 0.75), type = 6))
upper <- q[2] + 1.5 * (q[2] - q[1]) # the all-hubs upper fence from Part C
png("hub_boxplots_r.png", width = 700, height = 400)
b <- boxplot(Latency_Minutes ~ Hub, data = valid, horizontal = TRUE,
             col = "gray90", xlab = "Fulfillment latency (minutes)", ylab = "",
             main = "Valid latency by hub")
abline(v = upper, lty = 2)
dev.off()
colnames(b$stats) <- b$names
rownames(b$stats) <- c("lower whisker", "lower hinge", "median",
                      "upper hinge", "upper whisker")

print(b$stats)
# R draws the box from Tukey hinges, so box edges can differ slightly from QUARTILE.EXC.
# Compare: exclusive Q1, median and Q3 for each hub, as in Part D.
exc <- sapply(split(valid$Latency_Minutes, valid$Hub),
              function(x) quantile(x, c(0.25, 0.5, 0.75), type = 6))
print(exc)
```

Topic 5 · Where do the fence flags sit? Extend

Same question in Excel → SPSS → Python → R.

Excel Parts C, D

In a *copy* of your workbook, add a new sheet. **Step 1.** Type Hub A , Hub B and Hub C in A2:A4. **Step 2.** In B2 type `=COUNTIFS(Orders[Hub],A2,Orders[Latency_Minutes], "<>999",Orders[Latency_Minutes], ">"&'IN-CLASS_Lab Part C'!$B$7)` and fill down to B4. **Step 3.** In C2 type `=B2/'IN-CLASS_Lab Part D'!B7`, fill down to C4, and format column C as Percentage. Part D rows 7 to 9 hold the valid counts for Hub A to Hub C, so the fill lines up. **Step 4.** Check that B2 + B3 + B4 equals Part C B9, the 829 flagged records. Then chart column C as a bar chart with the axis starting at 0%.

SPSS Parts C, D

Flags every valid order above the Part C upper fence, then crosstabs the flag by hub with row percentages. Is the tail spread across hubs or concentrated in one?

New code: paste it at the end of LAB01_SPSS.sps , make sure the setup has run this session, then select only the pasted lines and choose **Run > Selection.**

```
* Extension: which hubs hold the orders above the upper Tukey fence .  
* Why: a count of flags is more useful when you know where the flags sit .  
* How: 49 is the Part C B7 upper fence; 999 is user-missing, so those rows get no flag .  
* Run: Run H0 first, then select this block and choose Run > Selection .
```

```
DATASET ACTIVATE OrdersData.  
TITLE 'Extension Fence Flags by Hub'.  
COMPUTE AboveFence=(Latency_Minutes > 49).  
VALUE LABELS AboveFence 0 'At or below fence' 1 'Above fence'.  
EXECUTE.  
CROSSTABS  
  /TABLES=Hub BY AboveFence  
  /CELLS=COUNT ROW.
```

Counts the flagged orders in each hub, then divides by that hub's own valid orders. The bar chart `fence_flags_by_hub.png` starts its axis at zero. The All Hubs row should match Part C B9.

```
# Extension: which hubs hold the orders above the upper Tukey fence? (Parts C and D)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("orders_story.csv", dtype={"OrderID": str, "CustomerID": str, "Hub": str})
valid = df[(df["Latency_Minutes"] != 999) & df["Latency_Minutes"].notna()].copy()
q1, q3 = np.percentile(valid["Latency_Minutes"], [25, 75], method="weibull")
upper = q3 + 1.5 * (q3 - q1) # the Part C B7 fence, all hubs
valid["AboveFence"] = valid["Latency_Minutes"] > upper

counts = pd.crosstab(valid["Hub"], valid["AboveFence"], margins=True, margins_name="All Hubs")
shares = pd.crosstab(valid["Hub"], valid["AboveFence"], normalize="index") * 100
print(f"Upper fence = {upper:.1f} min")
print(counts.to_string())
print(shares.round(1).to_string()) # row percent within each hub

ax = shares[True].plot.bar(color="gray", rot=0)
ax.set_ylim(0, 100) # an amount axis starts at zero
ax.set_ylabel("Percent of the hub's valid orders")
ax.set_title("Share of each hub above the all-hubs fence", loc="left")
plt.savefig("fence_flags_by_hub.png", dpi=120, bbox_inches="tight")
print("Saved fence_flags_by_hub.png.")
```

R Parts C, D

The same count and row-percent tables in base R, with the bar chart saved as `fence_flags_by_hub_r.png`. The Sum row should match Part C B9.

```
# Extension: which hubs hold the orders above the upper Tukey fence? (Parts C and D)
df <- read.csv("orders_story.csv",
               colClasses = c(OrderID = "character", CustomerID = "character", Hub = "character"))
valid <- df[!is.na(df$Latency_Minutes) & df$Latency_Minutes != 999, ]
q <- unname(quantile(valid$Latency_Minutes, c(0.25, 0.75), type = 6)) # QUANTILE.EXC
upper <- q[2] + 1.5 * (q[2] - q[1]) # the Part C B7 fence, all hubs
above <- factor(valid$Latency_Minutes > upper, levels = c(FALSE, TRUE),
               labels = c("At or below", "Above fence"))
tab <- table(Hub = valid$Hub, Fence = above)
cat("Upper fence =", sprintf("%.1f", upper), "min\n")
print(addmargins(tab, 1)) # the Sum row should match Part C B9
share <- round(100 * prop.table(tab, 1), 1) # row percent within each hub
print(share)
png("fence_flags_by_hub_r.png", width = 600, height = 400)
barplot(share[, "Above fence"], ylim = c(0, 100), col = "gray",
       ylab = "Percent of the hub's valid orders",
       main = "Share of each hub above the all-hubs fence")
dev.off()
```